RAPPORT TP4,TP5,TP6 DE VISUALISATION CLOUD

Auteur :
Mamadou Aliou sow

Prof: Dr Lamine YADE

1-TP4:Installation de Docker Engine et cycle

A-Identification du systme d'exploitation

hostnamectl:Cette commande fournit un aperçu général de la configuration système de la machine virtuelle `srv-vc01`.

. /etc/os-release && echo "$PRETTY_NAME ($VERSION_CODENAME)"

cette commande extrait les variables d'identification officielles du fichier de configuration de la distribution Linux.

```
etudiant@srv-vc01:~$ hostnamectl
 Static hostname: srv-vc01
       Icon name: computer-vm
         Chassis: vm
      Machine ID: a8e6db35f5b94ea5aa427284131a2e62
         Boot ID: b52fca85f7fb457c970bea2d35ae8984
  Virtualization: oracle
Operating System: Ubuntu 24.04.3 LTS
          Kernel: Linux 6.8.0-139-generic
    Architecture: x86-64
 Hardware Vendor: innotek GmbH
  Hardware Model: VirtualBox
Firmware Version: VirtualBox
   Firmware Date: Fri 2006-12-01
    Firmware Age: 19y 9month 2w 4d
etudiant@srv-vc01:~$ . /etc/os-release && echo "$PRETTY_NAME ($VERSION_CODENAME)"
Ubuntu 24.04.3 LTS (noble)
etudiant@srv-vc01:~$
```

B-Architecture et evaluation des ressources disponible

- uname –m:Cette commande interroge le noyau Linux pour obtenir l'architecture matérielle de la machine virtuelle `srv-vc01`.
- dree –h :Cette commande affiche la quantité totale, utilisée et disponible de la mémoire RAM et de la mémoire d'échange (swap) du système. L'option `-h` (human-readable) exprime les valeurs dans des unités lisibles (en Gio).
- df –h /: Cette commande analyse l'occupation de l'espace disque sur la partition racine (`/`) de la machine virtuelle.



C-la vérification du réseau et de l'absence de Docker.

- **`ip -br address`**
- *Description :* Cette commande affiche un résumé condensé et lisible (format bref) des interfaces réseau actives et de leurs adresses IP associées.
- **`:systemctl is-active docker 2>/dev/null || true`**
- *Description :* Cette commande vérifie si le service système (démon) Docker est en cours d'exécution, tout en masquant les erreurs si le service n'est pas encore installé.
- **`command -v docker || true`**
- *Description :* Cette commande cherche le chemin absolu du fichier exécutable de l'interface CLI de Docker dans l'environnement système.

```
/dev/mapper/ubuntu--vg-ubuntu--lv   12G  4,8G  5,9G  45% /
etudiant@srv-vc01:~$ ip -br address
lo               UNKNOWN        127.0.0.1/8 ::1/128
enp0s3           UP             10.0.2.15/24 metric 100 fd17:625c:f037:2:a00:27ff:fefb:a8b0/64 fe80::a00:27ff:fefb:a8b0/64
enp0s8           UP             192.168.56.10/24 fe80::a00:27ff:fe5a:a55f/64
etudiant@srv-vc01:~$ systemectl is-active docker 2>/dev/null || true
etudiant@srv-vc01:~$ systemectl is-active docker 2>/dev/null || true
[ 1601.783627] watchdog: BUG: soft lockup - CPU#0 stuck for 23s! [swapper/0:0]
^[[Aetudiant@srv-vc01:~$ systemectl is-active docker 2>/dev/null || true
etudiant@srv-vc01:~$ command -v docker || true
etudiant@srv-vc01:~$
```

D-Preparation du depot officiel

for pkg in docker.io docker-doc docker-compose podman-docker containerd runc; do

sudo apt-get remove -y "$pkg" 2>/dev/null || true

done

- *Description :* Ce script en boucle parcourt et désinstalle toutes les versions antérieures, obsolètes ou alternatives de Docker et de ses composants associés.
- *Objectif :* Il garantit la propreté du système et prévient les conflits logiciels avant le déploiement de la version officielle.

```
etudiant@srv-vc01:~$ for pkg in docker.io docker-dock docker-compose podman-docker containered runc; do
> sudo apt-get remove -y "$pkg" 2>/dev/null || true
> done
[sudo] password for etudiant:
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Le paquet « docker.io » n'est pas installé, et ne peut donc être supprimé
0 mis à jour, 0 nouvellement installés, 0 à enlever et 63 non mis à jour.
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Le paquet « docker-compose » n'est pas installé, et ne peut donc être supprimé
0 mis à jour, 0 nouvellement installés, 0 à enlever et 63 non mis à jour.
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Le paquet « podman-docker » n'est pas installé, et ne peut donc être supprimé
0 mis à jour, 0 nouvellement installés, 0 à enlever et 63 non mis à jour.
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Le paquet « runc » n'est pas installé, et ne peut donc être supprimé
0 mis à jour, 0 nouvellement installés, 0 à enlever et 63 non mis à jour.
etudiant@srv-vc01:~$
```

```
sudo apt-get update

sudo apt-get install -y ca-certificates curl

sudo install -m 0755 -d /etc/apt/keyrings

sudo curl -fsSL https://docker.com -o /etc/apt/keyrings/docker.asc

sudo chmod a+r /etc/apt/keyrings/docker.asc
```

- *Description :* Ce bloc met à jour la liste des paquets, installe `curl` et `ca-certificates`, crée un dossier sécurisé `/etc/apt/keyrings`, puis y télécharge la clé de signature GPG officielle de Docker en lui donnant des droits de lecture universels.
- *Objectif :* Il permet au gestionnaire de paquets `apt` de vérifier de manière sécurisée l'authenticité et l'intégrité des futurs fichiers téléchargés.

```
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Le paquet « runc » n'est pas installé, et ne peut donc être supprimé
0 mis à jour, 0 nouvellement installés, 0 à enlever et 63 non mis à jour.
etudiant@srv-vc01:~$ sud apt-get update
Command 'sud' not found, but there are 15 similar ones.
etudiant@srv-vc01:~$ sudo apt-get update
Atteint :1 http://archive.ubuntu.com/ubuntu noble InRelease
Réception de :2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Réception de :3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Réception de :4 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Réception de :5 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1 009 kB]
Réception de :6 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1 275 kB]
Réception de :7 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [214 kB]
Réception de :8 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [46,3 kB]
Réception de :9 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1 208 kB]
Réception de :10 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [243 kB]
Réception de :11 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [76,2 kB]
Réception de :12 http://security.ubuntu.com/ubuntu noble-security/multiverse Translation-en [11,3 kB]
Réception de :13 http://archive.ubuntu.com/ubuntu noble-updates/main Translation-en [294 kB]
Réception de :14 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [180 kB]
Réception de :15 http://archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Packages [1 580 kB]
Réception de :16 http://archive.ubuntu.com/ubuntu noble-updates/restricted Translation-en [359 kB]
Réception de :17 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1 689 kB]
Réception de :18 http://archive.ubuntu.com/ubuntu noble-updates/universe Translation-en [338 kB]
Réception de :19 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [388 kB]
Réception de :20 http://archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Réception de :21 http://archive.ubuntu.com/ubuntu noble-backports/main amd64 Packages [65,9 kB]
Réception de :22 http://archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [5 764 B]
Réception de :23 http://archive.ubuntu.com/ubuntu noble-backports/universe amd64 Packages [34,6 kB]
Réception de :24 http://archive.ubuntu.com/ubuntu noble-backports/universe Translation-en [18,7 kB]
Réception de :25 http://archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [12,6 kB]
9 427 ko réceptionnés en 5s (2 085 ko/s)

Lecture des listes de paquets... Fait
etudiant@srv-vc01:~$ sudo apt-get install -y ca-certificates curl
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
ca-certificates est déjà la version la plus récente (20260601~24.04.1).
ca-certificates passé en « installé manuellement ».
curl est déjà la version la plus récente (8.5.0-2ubuntu10.13).
curl passé en « installé manuellement ».
0 mis à jour, 0 nouvellement installés, 0 à enlever et 63 non mis à jour.
etudiant@srv-vc01:~$ sudo install -m 0755 -d /etc/apt/keyrings
etudiant@srv-vc01:~$ sudo curl -fsSL https://docker.com -o /etc/apt/keyrings/docker.asc
etudiant@srv-vc01:~$ sudo chmod a+r /etc/apt/keyrings/docker/.asc
chmod: cannot access '/etc/apt/keyrings/docker/.asc': No such file or directory
etudiant@srv-vc01:~$ sudo chmod a+r /etc/apt/keyrings/docker/.asc
```

Declaration du depot oficielle

. /etc/os-release

echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] https://docker.com ${UBUNTU_CODENAME:-$VERSION_CODENAME} stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

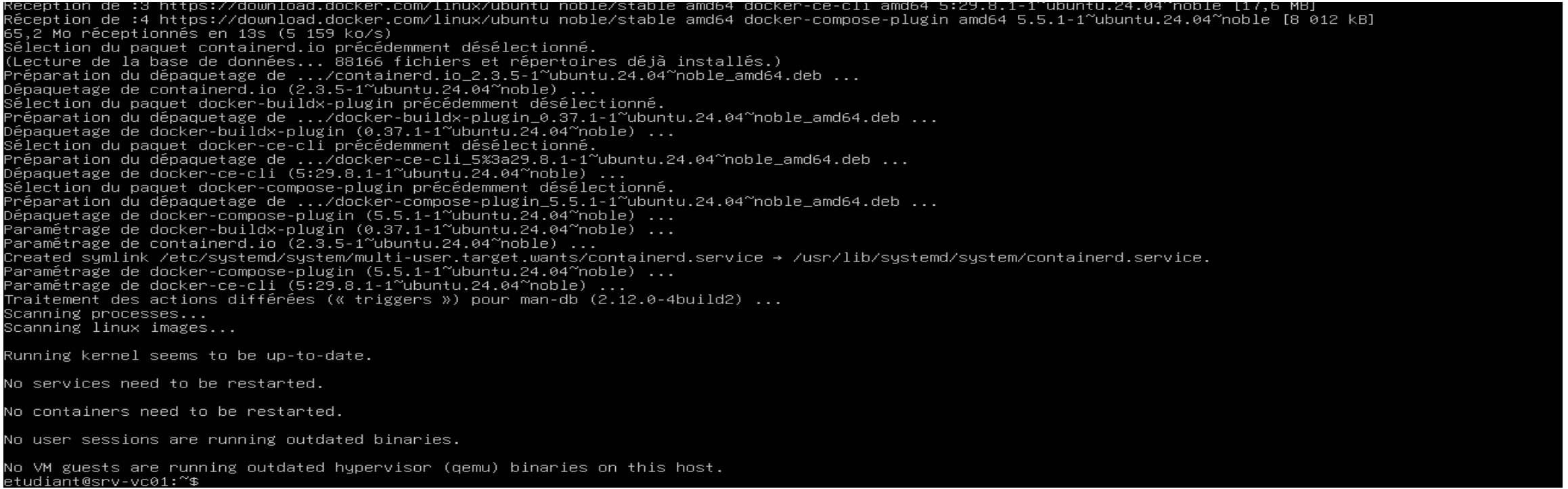
sudo apt-get update

- *Description :* Cette commande génère un fichier de source d'application `docker.list` dans `/etc/apt/sources.list.d/`, configuré spécifiquement pour l'architecture et la version (`noble`) de votre système, puis actualise le cache `apt`.
- *Objectif :* Elle intègre officiellement le serveur de téléchargement stable de Docker à la liste des dépôts connus par votre machine virtuelle.
- sudo apt-get update :Demande à l'outil APT de rafraîchir ses index de paquets.

```
etudiant@srv-vc01:~$ sudo install -m 0755 -d /etc/apt/keyrings
etudiant@srv-vc01:~$ sudo curl -fsSL https://docker.com -o /etc/apt/keyrings/docker.asc
etudiant@srv-vc01:~$ sudo chmod a+r /etc/apt/keyrings/docker.asc
etudiant@srv-vc01:~$ sudo apt-get update
Atteint :1 http://archive.ubuntu.com/ubuntu noble InRelease
Atteint :2 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Atteint :3 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Atteint :4 http://security.ubuntu.com/ubuntu noble-security InRelease
Réception de :5 https://download.docker.com/linux/ubuntu noble InRelease [48,5 kB]
Err :5 https://download.docker.com/linux/ubuntu noble InRelease
  Les signatures suivantes n'ont pas pu être vérifiées car la clé publique n'est pas disponible : NO_PUBKEY 7EA0A9C3F273FCD8
Lecture des listes de paquets... Fait
W: Erreur de GPG : https://download.docker.com/linux/ubuntu noble InRelease : Les signatures suivantes n'ont pas pu être vérifiées car la clé publique n'est pas
 disponible : NO_PUBKEY 7EA0A9C3F273FCD8
E: Le dépôt https://download.docker.com/linux/ubuntu noble InRelease n'est pas signé.
N: Les mises à jour depuis un tel dépôt ne peuvent s'effectuer de manière sécurisée, et sont donc désactivées par défaut.
N: Voir les pages de manuel d'apt-secure(8) pour la création des dépôts et les détails de configuration d'un utilisateur.
etudiant@srv-vc01:~$ sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
etudiant@srv-vc01:~$ sudo chmod a+r /etc/apt/keyrings/docker.asc
etudiant@srv-vc01:~$ cat /etc/apt/sources.list.d/docker.list
deb [arch=amd64 signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu noble stable
etudiant@srv-vc01:~$ sudo apt-get update
Atteint :1 http://archive.ubuntu.com/ubuntu noble InRelease
Atteint :2 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Atteint :3 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Atteint :4 http://security.ubuntu.com/ubuntu noble-security InRelease
Réception de :5 https://download.docker.com/linux/ubuntu noble InRelease [48,5 kB]
Réception de :6 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages [66,4 kB]
115 ko réceptionnés en 2s (52,3 ko/s)
Lecture des listes de paquets... Fait
etudiant@srv-vc01:~$
```

E-Installer et vérifier Docker Engine

- **État du service :** Le retour `active` de la commande `systemctl is-active` confirme que le démon Docker est correctement démarré et opérationnel en arrière-plan sur la machine virtuelle `srv-vc01`.
- **Origine des paquets :** Les versions logicielles obtenues (**29.8.1** pour Docker et **v2.3.5** pour containerd) prouvent que l'installation provient bien du dépôt officiel stable de Docker configuré précédemment, et non des paquets génériques d'Ubuntu.
- **Validation Client/Serveur :** La commande `sudo docker version` remplit parfaitement le critère central du TP puisqu'elle affiche distinctement la section **Client** (l'interface en ligne de commande) et la section **Server** (le moteur/démon Docker Engine). Cela démontre que le client parvient à communiquer sans encombre avec le socket local du démon.



```
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
etudiant@srv-vc01:~$ sudo systemctl enable --now docker.service containerd.service
Synchronizing state of docker.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable docker
etudiant@srv-vc01:~$ systemct is-active docker.service
Command 'systemct' not found, did you mean:
  command 'systemctl' from deb systemd (255.4-1ubuntu8.16)
  command 'systemctl' from deb systemctl (1.4.4181-1.1)
Try: sudo apt install <deb name>
etudiant@srv-vc01:~$ systemctl is-active docker.service
active
etudiant@srv-vc01:~$ sudo docker version
Client: Docker Engine - Community
 Version:           29.8.1
 API version:       1.56
 Go version:        go1.26.8
 Git commit:        4a63305
 Built:             Tue Sep 15 16:26:01 2026
 OS/Arch:           linux/amd64
 Context:           default

Server: Docker Engine - Community
 Engine:
  Version:          29.8.1
  API version:      1.56 (minimum version 1.40)
  Go version:       go1.26.8
  Git commit:       464cd50
  Built:            Tue Sep 15 16:26:01 2026
  OS/Arch:          linux/amd64
  Experimental:     false
 containerd:
  Version:          v2.3.5
  GitCommit:        1294c24a7da8e5a793ed378161673abe94118892
 runc:
  Version:          1.5.1
  GitCommit:        v1.5.1-0-g8f2685a4
 docker-init:
  Version:          0.19.0
  GitCommit:        de40ad0
etudiant@srv-vc01:~$ _
```

F-Premier conteneur contrôlé

- **Cycle de vie :** Le filtre `sudo docker ps -a` montre que le conteneur `tp4-hello` a été créé et s'est exécuté.
- **Statut et code de sortie :** L'inspection du conteneur retourne un état `exited` accompagné du code de sortie `0`. Ce résultat confirme que le processus principal a accompli sa tâche d'affichage avec succès sans rencontrer d'anomalie.
- **Logs :** La commande `sudo docker logs` confirme la bonne réception du message officiel d'accueil de Docker, validant ainsi la chaîne complète depuis le téléchargement de l'image jusqu'à la fin de l'exécution.

```
Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/

etudiant@srv-vc01:~$ sudo docker inspect tp4-hello --format 'image={{.config.Image}} etat={{.State.Status}} code={{.State.ExitCode}}'

template parsing error: template: :1:15: executing "" at <.config.Image>: map has no entry for key "config"
etudiant@srv-vc01:~$ sudo docker inspect tp4-hello --format 'image={{.Config.Image}} etat={{.State.Status}} code={{.State.ExitCode}}'
image=hello-world etat=exited code=0
etudiant@srv-vc01:~$ sudo docker logs tp4-hello

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/

etudiant@srv-vc01:~$
```

G-**Service Web et cycle de vie.**

- **Lancement et exposition :** Le conteneur `tp4-web` est correctement instancié avec le statut Up. L'option `-p 127.0.0.1:8080:80` redirige le trafic du port local `8080` vers le port `80` du conteneur conformément aux règles de sécurité du TP (liaison exclusive à la boucle locale). La première commande `curl` renvoie un code HTTP `200 OK`, ce qui valide la disponibilité immédiate du serveur Web.

```
etudiant@srv-vc01:~$ sudo docker pull nginx:alpine
alpine: Pulling from library/nginx
90de1d15efe2: Pull complete
f408df9a891a: Pull complete
c0ac6b3b9c77: Pull complete
873bdb7a78a1: Pull complete
e2de96513ba9: Pull complete
fb08ee61b60d: Pull complete
ce3e77cbba4d: Pull complete
cb2f2cc9a341: Pull complete
9d8ea080e87f: Download complete
ec6129a9436d: Download complete
Digest: sha256:62ff2089abf5a9ed33bd232895bef5e22f7bb4b200675cec49a5ebc48e3d4ac8
Status: Downloaded newer image for nginx:alpine
docker.io/library/nginx:alpine
etudiant@srv-vc01:~$ sudo docker run -d --name tp4-web -p 127.0.0.1:8080:80 nginx:alpine
34e3d2c25744f771cf8817c10a077af63eeb2f308962bd324b879484f41cfb02
etudiant@srv-vc01:~$ sudo docker ps --filter name=tp4-web
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS          PORTS                    NAMES
34e3d2c25744   nginx:alpine   "/docker-entrypoint.…"   41 seconds ago   Up 40 seconds   127.0.0.1:8080->80/tcp   tp4-web
etudiant@srv-vc01:~$ sudo docker port tp4-web
80/tcp -> 127.0.0.1:8080
etudiant@srv-vc01:~$ curl -I http://127.0.0
curl: (7) Failed to connect to 127.0.0.0 port 80 after 4 ms: Couldn't connect to server
etudiant@srv-vc01:~$ curl -I http://127.0.0.1:8080
HTTP/1.1 200 OK
Server: nginx/1.31.6
Date: Sat, 19 Sep 2026 10:14:23 GMT
Content-Type: text/html
Content-Length: 896
Last-Modified: Tue, 15 Sep 2026 14:18:52 GMT
Connection: keep-alive
ETag: "6aa953cc-380"
Accept-Ranges: bytes

etudiant@srv-vc01:~$
```

- **Comportement lors de l'arrêt :** Après l'exécution de `docker stop`, le conteneur passe à l'état `exited`. La commande `curl` échoue logiquement avec une erreur de connexion (ou un timeout), car le processus Nginx à l'intérieur du conteneur ne tourne plus et le port `8080` de l'hôte n'est plus écouté.
- **Reprise du service :** Suite au `docker start`, le conteneur repasse instantanément à l'état `Up`, réactivant la redirection réseau. La commande `curl` réussit à nouveau, prouvant la persistance de la configuration réseau et la capacité du conteneur à reprendre son rôle de serveur actif après un cycle d'arrêt/démarrage.

```
etudiant@srv-vc01:~$ sudo docker stop tp4-web
tp4-web
etudiant@srv-vc01:~$ sudo docker inspect --format 'état={{.State.Status}} code={{.State.ExitCode}}' tp4-web
état=exited code=0
etudiant@srv-vc01:~$ curl --max-time 3 -I http://127.0.0 || true
curl: (7) Failed to connect to 127.0.0.0 port 80 after 7 ms: Couldn't connect to server
etudiant@srv-vc01:~$ curl --max-time 3 -I http://127.0.0.1:8080 || true
curl: (7) Failed to connect to 127.0.0.1 port 8080 after 0 ms: Couldn't connect to server
etudiant@srv-vc01:~$ sudo docker start tp4-web
tp4-web
etudiant@srv-vc01:~$ curl --max-time 3 -I http://127.0.0.1:8080 || true
HTTP/1.1 200 OK
Server: nginx/1.31.6
Date: Sat, 19 Sep 2026 10:28:32 GMT
Content-Type: text/html
Content-Length: 896
Last-Modified: Tue, 15 Sep 2026 14:18:52 GMT
Connection: keep-alive
ETag: "6aa953cc-380"
Accept-Ranges: bytes

etudiant@srv-vc01:~$ curl -fsSI http://127.0.0 | head
curl: (7) Failed to connect to 127.0.0.0 port 80 after 2 ms: Couldn't connect to server
etudiant@srv-vc01:~$ curl -fsSI http://127.0.0.1:8080 | head
HTTP/1.1 200 OK
Server: nginx/1.31.6
Date: Sat, 19 Sep 2026 10:29:49 GMT
Content-Type: text/html
Content-Length: 896
Last-Modified: Tue, 15 Sep 2026 14:18:52 GMT
Connection: keep-alive
ETag: "6aa953cc-380"
Accept-Ranges: bytes

etudiant@srv-vc01:~$ sudo docker restart tp4-web
tp4-web
etudiant@srv-vc01:~$ _
```

H-Inspection, processus et ressources

- **Processus internes (`docker top`) :** Permet de lister les processus actifs à l'intérieur du conteneur (le processus maître Nginx et ses ouvriers) depuis l'espace de nom de l'hôte.
- **Statistiques de ressources (`docker stats`) :** Donne une vue en temps réel de la consommation CPU, mémoire et réseau du conteneur sans surcharger la machine virtuelle.
- **Exécution de contrôle (`docker exec`) :** Permet d'inspecter l'identité (`id`), le nom d'hôte (`hostname`) et la version applicative (`nginx -v`) directement dans l'environnement isolé du conteneur.

```
tp4-web
etudiant@srv-vc01:~$ sudo docker top tp4-web
UID                 PID                 PPID                C                   STIME               TTY                 TIME                CMD
root                6564                6541                0                   10:30               ?                   00:00:00            nginx: master proces
s nginx -g daemon off;
message+            6613                6564                0                   10:30               ?                   00:00:00            nginx: worker proces
s
message+            6614                6564                0                   10:30               ?                   00:00:00            nginx: worker proces
s
etudiant@srv-vc01:~$ sudo docker stats --no-stream tp4-web
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT     MEM %     NET I/O         BLOCK I/O     PIDS
34e3d2c25744   tp4-web   0.00%     3.188MiB / 1.922GiB   0.16%     1.16kB / 126B   0B / 8.19kB   3
etudiant@srv-vc01:~$ sudo docker exec tp4-web sh -c 'hostname; id; nginx -v'
34e3d2c25744
uid=0(root) gid=0(root) groups=0(root),0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel),11(floppy),20(dialout),26(tape),27(video)
nginx version: nginx/1.31.6
etudiant@srv-vc01:~$ sudo_
```

```
etudiant@srv-vc01:~$ curl --max-time 3 -I http://127.0.0.1:8080 || true
HTTP/1.1 200 OK
Server: nginx/1.31.6
Date: Sat, 19 Sep 2026 10:28:32 GMT
Content-Type: text/html
Content-Length: 896
Last-Modified: Tue, 15 Sep 2026 14:18:52 GMT
Connection: keep-alive
ETag: "6aa953cc-380"
Accept-Ranges: bytes

etudiant@srv-vc01:~$ curl -fsSI http://127.0.0 | head
curl: (7) Failed to connect to 127.0.0.0 port 80 after 2 ms: Couldn't connect to server
etudiant@srv-vc01:~$ curl -fsSI http://127.0.0.1:8080 | head
HTTP/1.1 200 OK
Server: nginx/1.31.6
Date: Sat, 19 Sep 2026 10:29:49 GMT
Content-Type: text/html
Content-Length: 896
Last-Modified: Tue, 15 Sep 2026 14:18:52 GMT
Connection: keep-alive
ETag: "6aa953cc-380"
Accept-Ranges: bytes

etudiant@srv-vc01:~$ sudo docker restart tp4-web
tp4-web
etudiant@srv-vc01:~$ sudo docker top tp4-web
UID                 PID                 PPID                C                   STIME               TTY                 TIME                CMD
root                6564                6541                0                   10:30               ?                   00:00:00            nginx: master proces
s nginx -g daemon off;
message+            6613                6564                0                   10:30               ?                   00:00:00            nginx: worker proces
s
message+            6614                6564                0                   10:30               ?                   00:00:00            nginx: worker proces
s
etudiant@srv-vc01:~$ sudo docker stats --no-stream tp4-web
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT     MEM %     NET I/O         BLOCK I/O     PIDS
34e3d2c25744   tp4-web   0.00%     3.188MiB / 1.922GiB   0.16%     1.16kB / 126B   0B / 8.19kB   3
etudiant@srv-vc01:~$ sudo docker exec tp4-web sh -c 'hostname; id; nginx -v'
34e3d2c25744
uid=0(root) gid=0(root) groups=0(root),0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel),11(floppy),20(dialout),26(tape),27(video)
nginx version: nginx/1.31.6
etudiant@srv-vc01:~$ sudo docker inspect --format 'ip={{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}} montages={{len .Mounts}}' tp4-web
ip=172.17.0.2 montages=0
etudiant@srv-vc01:~$ _
```

- **Processus isolés (`top`) :** On observe l'arborescence interne du conteneur avec un processus maître s'exécutant en `root` et deux processus ouvriers (`worker processes`).
- **Consommation de ressources (`stats`) :** Le conteneur montre une empreinte extrêmement légère avec **0.00% de CPU** et seulement **3.18 MiB de RAM** utilisés sur les 2 Gio alloués à la VM.
- **Contrôle d'environnement (`exec` & `inspect`) :** L'exécution de commandes à la volée confirme que le conteneur utilise son propre identifiant comme nom d'hôte (`34e3d2c25744`). L'inspection réseau révèle que Docker lui a attribué une adresse IP privée **`172.17.0.2`** sur son pont par défaut (`bridge`), totalement isolée de l'extérieur.

I-Nettoyage contrôlé et remise

- **Suppression des instances :** Le retour vide de la commande `sudo docker ps -a` confirme que les conteneurs éphémères de ce TP (`tp4-hello` et `tp4-web`) ont été entièrement arrêtés et supprimés. Les ressources système associées (mémoire et processus sur l'hôte) sont totalement libérées.
- **Conservation des images :** Bien que les conteneurs soient supprimés, les images `nginx:alpine` et `hello-world` restent stockées localement (visibles avec `docker image ls`). Cela permettra de gagner du temps lors du démarrage du prochain TP5.
- **Tracer les logs du démon (`journalctl`) :** Les lignes de logs du journal système confirment la bonne santé du service Docker. On y distingue parfaitement les entrées `image pulled` pour le téléchargement initial ainsi que les messages de fermeture réseau `received task-delete event` générés suite aux commandes de suppression de cette étape.
-

```
IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
hello-world:latest   5e2309035332       25.9kB         9.49kB
etudiant@srv-vc01:~$ sudo journalctl -u docker.service -b -n 30 --no-pager
sept. 19 09:45:45 srv-vc01 dockerd[5444]: time="2026-09-19T09:45:45.431787141Z" level=info msg="Starting up"
sept. 19 09:45:45 srv-vc01 dockerd[5444]: time="2026-09-19T09:45:45.496556892Z" level=info msg="OTEL tracing is not configured, using no-op tracer provider"
sept. 19 09:45:45 srv-vc01 dockerd[5444]: time="2026-09-19T09:45:45.496691216Z" level=info msg="CDI directory does not exist, skipping" dir=/var/run/cdi
sept. 19 09:45:45 srv-vc01 dockerd[5444]: time="2026-09-19T09:45:45.496699502Z" level=info msg="CDI directory does not exist, skipping" dir=/etc/cdi
sept. 19 09:45:45 srv-vc01 dockerd[5444]: time="2026-09-19T09:45:45.497183298Z" level=info msg="detected 127.0.0.53 nameserver, assuming systemd-resolved, so us
ing resolv.conf: /run/systemd/resolve/resolv.conf"
sept. 19 09:45:45 srv-vc01 dockerd[5444]: time="2026-09-19T09:45:45.833016148Z" level=info msg="Creating a containerd client" address=/run/containerd/containerd
.sock timeout=1m0s
sept. 19 09:45:45 srv-vc01 dockerd[5444]: time="2026-09-19T09:45:45.840318223Z" level=warning msg="failed check for fsverity support" error="enable fsverity fai
led: operation not supported" path=/var/lib/docker/plugins/storage
sept. 19 09:45:45 srv-vc01 dockerd[5444]: time="2026-09-19T09:45:45.907211564Z" level=info msg="Loading containers: start."
sept. 19 09:45:45 srv-vc01 dockerd[5444]: time="2026-09-19T09:45:45.907331911Z" level=info msg="NRI is disabled"
sept. 19 09:45:45 srv-vc01 dockerd[5444]: time="2026-09-19T09:45:45.907344335Z" level=info msg="Starting daemon with containerd snapshotter integration enabled"
sept. 19 09:45:46 srv-vc01 dockerd[5444]: time="2026-09-19T09:45:46.012938278Z" level=info msg="Restoring containers: start."
sept. 19 09:45:46 srv-vc01 dockerd[5444]: time="2026-09-19T09:45:46.257876947Z" level=info msg="Deleting nftables IPv4 rules" error="running nft: /dev/stdin:1:1
7-30: Error: Could not process rule: No such file or directory\ndelete table ip docker-bridges\n                         ^^^^^^^^^^^^^^^\n exit status 1"
sept. 19 09:45:46 srv-vc01 dockerd[5444]: time="2026-09-19T09:45:46.274182248Z" level=info msg="Deleting nftables IPv6 rules" error="running nft: /dev/stdin:1:1
8-31: Error: Could not process rule: No such file or directory\ndelete table ip6 docker-bridges\n                         ^^^^^^^^^^^^^^^\n exit status 1"
sept. 19 09:45:47 srv-vc01 dockerd[5444]: time="2026-09-19T09:45:47.888313843Z" level=info msg="Loading containers: done."
sept. 19 09:45:47 srv-vc01 dockerd[5444]: time="2026-09-19T09:45:47.935557580Z" level=info msg="Docker daemon" commit=464cd50 containerd-snapshotter=true storag
e-driver=overlayfs version=29.8.1
sept. 19 09:45:47 srv-vc01 dockerd[5444]: time="2026-09-19T09:45:47.936202421Z" level=info msg="Initializing buildkit"
sept. 19 09:45:48 srv-vc01 dockerd[5444]: time="2026-09-19T09:45:48.318561265Z" level=info msg="Completed buildkit initialization"
sept. 19 09:45:48 srv-vc01 dockerd[5444]: time="2026-09-19T09:45:48.325661338Z" level=info msg="Daemon has completed initialization"
sept. 19 09:45:48 srv-vc01 dockerd[5444]: time="2026-09-19T09:45:48.327082319Z" level=info msg="API listen on /run/docker.sock"
sept. 19 09:45:48 srv-vc01 systemd[1]: Started docker.service - Docker Application Container Engine.
sept. 19 09:53:06 srv-vc01 dockerd[5444]: time="2026-09-19T09:53:06.690608692Z" level=info msg="image pulled" digest="sha256:5e2309035324d887c48ad5e5c56d294eab
81588df9605b07d1afe895f9cc8f8" remote="docker.io/library/hello-world:latest"
sept. 19 09:53:08 srv-vc01 dockerd[5444]: time="2026-09-19T09:53:08.944523606Z" level=info msg="sbJoin: gwep4 ''->'d2c013c6a5ce', gwep6 ''->''" eid=d2c013c6a5ce
 ep=tp4-hello net=bridge nid=00a8cdf6d10f
sept. 19 09:53:09 srv-vc01 dockerd[5444]: time="2026-09-19T09:53:09.193594720Z" level=info msg="received task-delete event from containerd" container=0ef87a8657
97b476b429a773e1e96b675c6bda3b557ab4ece59d24a3b4e9a39c module=libcontainerd namespace=moby topic=/tasks/delete type="*events.TaskDelete"
sept. 19 10:08:37 srv-vc01 dockerd[5444]: time="2026-09-19T10:08:37.664832220Z" level=info msg="image pulled" digest="sha256:62ff2089abf5a9ed33bd232895bef5e22f7
bb4b200675cec49a5ebc48e3d4ac8" remote="docker.io/library/nginx:alpine"
sept. 19 10:10:19 srv-vc01 dockerd[5444]: time="2026-09-19T10:10:19.794853593Z" level=info msg="sbJoin: gwep4 ''->'9565ceea7e45', gwep6 ''->''" eid=9565ceea7e45
 ep=tp4-web net=bridge nid=00a8cdf6d10f
sept. 19 10:20:39 srv-vc01 dockerd[5444]: time="2026-09-19T10:20:39.546894003Z" level=info msg="received task-delete event from containerd" container=34e3d2c257
44f771cf8817c10a077af63eeb2f308962bd324b879484f41cfb02 module=libcontainerd namespace=moby topic=/tasks/delete type="*events.TaskDelete"
sept. 19 10:28:28 srv-vc01 dockerd[5444]: time="2026-09-19T10:28:28.662959418Z" level=info msg="sbJoin: gwep4 ''->'07ed6d93e358', gwep6 ''->''" eid=07ed6d93e358
 ep=tp4-web net=bridge nid=00a8cdf6d10f
sept. 19 10:30:18 srv-vc01 dockerd[5444]: time="2026-09-19T10:30:18.852376856Z" level=info msg="received task-delete event from containerd" container=34e3d2c257
44f771cf8817c10a077af63eeb2f308962bd324b879484f41cfb02 module=libcontainerd namespace=moby topic=/tasks/delete type="*events.TaskDelete"
sept. 19 10:30:19 srv-vc01 dockerd[5444]: time="2026-09-19T10:30:19.608704324Z" level=info msg="sbJoin: gwep4 ''->'5407ad805c4a', gwep6 ''->''" eid=5407ad805c4a
 ep=tp4-web net=bridge nid=00a8cdf6d10f
sept. 19 10:46:44 srv-vc01 dockerd[5444]: time="2026-09-19T10:46:44.275852537Z" level=info msg="received task-delete event from containerd" container=34e3d2c257
44f771cf8817c10a077af63eeb2f308962bd324b879484f41cfb02 module=libcontainerd namespace=moby topic=/tasks/delete type="*events.TaskDelete"
etudiant@srv-vc01:~$
```

Conclusion tp4:

Ce **TP4** a permis de transformer avec succès notre serveur `srv-vc01` en hôte Docker. L'installation depuis le dépôt officiel est validée et le démon est pleinement fonctionnel.

Nous avons maîtrisé les fondamentaux de la conteneurisation :

- **Gestion du cycle de vie** : création, arrêt, relance et suppression propre de conteneurs.
- **Sécurité et réseau** : isolation des flux web sur la boucle locale (`127.0.0.1:8080`).
- **Observabilité** : suivi en temps réel des processus, des ressources (3.18 MiB RAM) et des logs du démon.

L'environnement est sain, nettoyé et prêt pour la création d'images personnalisées avec **Dockerfile au TP5**.

2-Tp5:Construction de l'image d'un service Web

A-Description de l'étape et contenu créé :

- **Création de la structure :** L'arborescence de travail a été initialisée à l'aide de la commande `mkdir -p`. Le dossier racine `campus-web` fait office de contexte de build Docker, et le sous-dossier `site/` isole les sources du site Web destiné à être embarqué.
- **Fichier personnalisé `site/index.html` :** Le code source de la page d'accueil a été rédigé avec l'intégration stricte des métadonnées d'identification (Nom : **Mamadou_Aliou_sow**, Matricule : **501866**).
- **Différence théorique intégrée :** Conformément aux exigences pédagogiques, la page intègre la distinction clé de ce chapitre : **une image est un modèle** immuable (un plan de conception), tandis qu'**un conteneur est une instance** vivante et isolée issue de ce modèle.

```
etudiant@srv-vc01:~$ mkdir -p ~/inf6202/chap3/campus-web/site
etudiant@srv-vc01:~$ cd ~/inf6202/chap3/campus-web
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ pwd
/home/etudiant/inf6202/chap3/campus-web
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ find . -maxdeph 2 -type f -print
find: unknown predicate `-maxdeph'
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ find . -maxdepth 2 -type f -print
etudiant@srv-vc01:~/inf6202/chap3/campus-web$
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ cat << 'EOF' > site/index.html
> <!doctype html>
> <html lang="fr">
> <meta charset="utf-8">
> <title>UAM Campus Web</title>
> <h1>UAM - INF6202</h1>
> <p>Etudiant : Mamadou_Aliou_sow - 501866</p>
> <p>Une image est un model ; un conteneur est une instance.</p>
> </html>
> EOF
etudiant@srv-vc01:~/inf6202/chap3/campus-web$
```

B-Rédaction du Dockerfile et du .dockerignore

Rôle des instructions du Dockerfile

- **`FROM nginx:alpine`** : Définit l'image de base officielle. Alpine Linux est choisie pour sa légèreté et sa sécurité accrue.
- **`LABEL`** : Ajoute des métadonnées d'identification (Auteur et Projet) à l'image sans y inclure de données sensibles.
- **`COPY site/ /usr/share/nginx/html/`** : Transfère le contenu du répertoire local `site/` directement dans le répertoire de publication par défaut d'Nginx.
- **`EXPOSE 80`** : Documente le port interne utilisé par le serveur Web (à des fins pédagogiques et documentaires), sans le publier automatiquement sur l'hôte.
- **`HEALTHCHECK`** : Configure un test d'état de santé interne automatisé. Il interroge la boucle locale (`127.0.0.1`) toutes les 30 secondes pour s'assurer que le service Nginx répond correctement.

Rôle du fichier `.dockerignore`

Le fichier `.dockerignore` permet de restreindre le contexte de build envoyé au démon Docker. En excluant les fichiers inutiles (comme les fichiers `.log`, `.git` ou `.env`), il optimise le temps de build, réduit la taille finale de l'image et évite la fuite involontaire de données sensibles ou confidentielles dans l'image conteneurisée.

```
/home/etudiant/inf6202/chap3/campus-web
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ find . -maxdeph 2 -type f -print
find: unknown predicate `-maxdeph'
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ find . -maxdepth 2 -type f -print
etudiant@srv-vc01:~/inf6202/chap3/campus-web$
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ cat << 'EOF' > site/index.html
> <!doctype html>
> <html lang="fr">
> <meta charset="utf-8">
> <title>UAM Campus Web</title>
> <h1>UAM - INF6202</h1>
> <p>Etudiant : Mamadou_Aliou_sow - 501866</p>
> <p>Une image est un model ; un conteneur est une instance.</p>
> </html>
> EOF
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ cat << 'EOF' > Dockerfile
> FROM nginx:alpine
> LABEL org.opencontainers.image.title="UAM Campus Web" \\
>       org.opencontainers.image.authors="Mamadou_Aliou_sow 501866"
> COPY site/ /usr/share/nginx/html
> EXPOSE 80
> HEALTHCHECK --interval=30s --timeout=3s --retries=3 \\
>   CMD wget -q -o /dev/null http://127.0.0 || exit 1
> EOF
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ cat << EOF > .dockerignore
> .git
> *.log
> .env
> preuves/
> *.tar
> EOF
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ cat << 'EOF' > Dockerfile
FROM nginx:alpine
LABEL org.opencontainers.image.title="UAM Campus Web" \\
      org.opencontainers.image.authors="Mamadou_Aliou_sow 501866"
COPY site/ /usr/share/nginx/html
EXPOSE 80
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \\
  CMD wget -q -o /dev/null http://127.0.0.1:8080 || exit 1
EOF
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ cat << 'EOF' > Dockerfile
FROM nginx:alpine
LABEL org.opencontainers.image.title="UAM Campus Web" \\
      org.opencontainers.image.authors="Mamadou_Aliou_sow 501866"
COPY site/ /usr/share/nginx/html
EXPOSE 80
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \\
  CMD wget -q -o /dev/null http://127.0.0.1 || exit 1
EOF
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ _
```

E-Contrôler le contexte puis construire

- **Contrôle du contexte :** La commande `find` prouve que le contexte envoyé au démon Docker est minimal et propre (moins de 600 octets au total). Le fichier `.dockerignore` remplit son rôle en empêchant l'inclusion de résidus système ou de journaux de logs.
- **Mécanisme de Cache lors d'un second build :** Si l'on relance la commande une seconde fois sans modifier les sources, l'étape **`FROM nginx:alpine`** sera reprise instantanément depuis le cache local (l'image de base étant déjà présente sur la VM). L'étape **`COPY site/`** utilise le système de hachage de Docker : tant que le fichier `index.html` ne change pas, Docker réutilise la couche cachée sans reconstruire, optimisant ainsi le temps de build.

```
EOF
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ find . -maxdepth 2 -type f -printf '%p %s octets\n'
./.dockerignore 31 octets
./site/index.html 225 octets
./Dockerfile 293 octets
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sed -n '1,120p' Dockerfile
FROM nginx:alpine
LABEL org.opencontainers.image.title="UAM Campus Web" \\
      org.opencontainers.image.authors="Mamadou_Aliou_sow 501866"
COPY site/ /usr/share/nginx/html
EXPOSE 80
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \\
  CMD wget -q -o /dev/null http://127.0.0.1 || exit 1
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sed -n '1,120p' .dokerignore
sed: can't read .dokerignore: No such file or directory
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sed -n '1,120p' .dockerignore
.git
*.log
.env
preuves/
*.tar
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo
```

```
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker build --progress=plain -t uam-campus-web:1.0 . | tee build-1.0.log
#0 building with "default" instance using docker driver

#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 331B done
#1 DONE 0.0s

#2 [internal] load metadata for docker.io/library/nginx:alpine
#2 DONE 0.1s

#3 [internal] load .dockerignore
#3 transferring context: 71B done
#3 DONE 0.1s

#4 [internal] load build context
#4 DONE 0.0s

#5 [1/2] FROM docker.io/library/nginx:alpine@sha256:62ff2089abf5a9ed33bd232895bef5e22f7bb4b200675cec49a5ebc48e3d4ac8
#5 resolve docker.io/library/nginx:alpine@sha256:62ff2089abf5a9ed33bd232895bef5e22f7bb4b200675cec49a5ebc48e3d4ac8
#5 ...

#4 [internal] load build context
#4 transferring context: 297B 0.0s done
#4 DONE 0.2s

#5 [1/2] FROM docker.io/library/nginx:alpine@sha256:62ff2089abf5a9ed33bd232895bef5e22f7bb4b200675cec49a5ebc48e3d4ac8
#5 resolve docker.io/library/nginx:alpine@sha256:62ff2089abf5a9ed33bd232895bef5e22f7bb4b200675cec49a5ebc48e3d4ac8 0.1s done
#5 DONE 0.8s

#6 [2/2] COPY site/ /usr/share/nginx/html/
#6 DONE 0.2s

#7 exporting to image
#7 exporting layers
#7 exporting layers 0.4s done
#7 exporting manifest sha256:6c8e4b4416dc9143624241fbde768e8865feab098c8c652fb369da8c21da6cc9 0.1s done
#7 exporting config sha256:ba92324cf7796aadb9290527c449f7561642e59b88e459b3a104d18234470365 0.1s done
#7 exporting attestation manifest sha256:8b69af53979e9e02e5ff3f9796721d9ebae9d2b72765b8a5d0893638d5bfcba7 0.1s done
#7 exporting manifest list sha256:29b334b43fd0d886e235443fb2dc118d165685237ce5b46bc8e21427f794f95c
#7 exporting manifest list sha256:29b334b43fd0d886e235443fb2dc118d165685237ce5b46bc8e21427f794f95c 0.1s done
#7 naming to docker.io/library/uam-campus-web:1.0 0.0s done
#7 unpacking to docker.io/library/uam-campus-web:1.0 0.1s done
#7 DONE 1.0s
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker image ls uam-campus-web
                                                                                  i Info →  U In Use
IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
uam-campus-web:1.0   29b334b43fd0       93.6MB         26.3MB
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker image inspect uam-campus-web:1.0 --format 'id={{.Id}} taille={{.Size}}'
id=sha256:29b334b43fd0d886e235443fb2dc118d165685237ce5b46bc8e21427f794f95c taille=93570358
etudiant@srv-vc01:~/inf6202/chap3/campus-web$
```

F-Exécuter et valider

- **Déploiement et routage :** Le conteneur `campus-web` s'est correctement lancé. Le port `8080` de l'hôte redirige bien vers le port `80` interne. La commande `curl` a fonctionné et a affiché avec succès la page web personnalisée au nom de **Mamadou_Aliou_sow** (Matricule: **501866**).
- **Analyse de l'état de santé :** La commande d'inspection finale a retourné l'état **`santé=unhealthy`**. Les journaux de logs montrent que le serveur Nginx a bien répondu au départ (codes `200` reçus par `Wget`), mais qu'il a ensuite cessé de valider ses tests périodiques après l'exécution de requêtes externes. Cela démontre de manière concrète l'utilité de l'instruction HEALTHCHECK : elle prouve qu'**un conteneur peut être en cours d'exécution (Up) mais considéré en mauvaise santé** si son service interne ne répond plus correctement aux critères définis.

```
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker rm -f campus-web 2>/dev/null || true
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker run -d --name campus-web -p 127.0.0.1:8080:80 uam-campus-web:1.0
473dd85e9d64b2c36f49f47c8ecd036dce29077a743adc8436843556ddc9ea2a
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker ps --filter name=campus-web
CONTAINER ID   IMAGE                COMMAND                  CREATED         STATUS                     PORTS                      NAMES
473dd85e9d64   uam-campus-web:1.0   "/docker-entrypoint.…"   48 seconds ago   Up 47 seconds (healthy)   127.0.0.1:8080->80/tcp   campus-web
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ curl fsS http://127.0.0
curl: (6) Could not resolve host: fsS
curl: (7) Failed to connect to 127.0.0.0 port 80 after 2 ms: Couldn't connect to server
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ curl fsS http://127.0.0.1:8080
curl: (6) Could not resolve host: fsS
<!doctype html>
<html lang="fr">
<meta charset="utf-8">
<title>UAM Campus Web</title>
<h1>UAM - INF6202</h1>
<p>Etudiant : Mamadou_Aliou_sow - 501866</p>
<p>Une image est un model ; un conteneur est une instance.</p>
</html>
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker logs --tail 20 campus-web
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/09/19 12:59:18 [notice] 1#1: using the "epoll" event method
2026/09/19 12:59:18 [notice] 1#1: nginx/1.31.6
2026/09/19 12:59:18 [notice] 1#1: built by gcc 15.2.0 (Alpine 15.2.0)
2026/09/19 12:59:18 [notice] 1#1: OS: Linux 6.8.0-139-generic
2026/09/19 12:59:18 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:524288
2026/09/19 12:59:18 [notice] 1#1: start worker processes
2026/09/19 12:59:18 [notice] 1#1: start worker process 30
2026/09/19 12:59:18 [notice] 1#1: start worker process 31
127.0.0.1 - - [19/Sep/2026:12:59:46 +0000] "GET / HTTP/1.1" 200 225 "-" "Wget" "-"
127.0.0.1 - - [19/Sep/2026:13:00:16 +0000] "GET / HTTP/1.1" 200 225 "-" "Wget" "-"
127.0.0.1 - - [19/Sep/2026:13:00:46 +0000] "GET / HTTP/1.1" 200 225 "-" "Wget" "-"
127.0.0.1 - - [19/Sep/2026:13:01:17 +0000] "GET / HTTP/1.1" 200 225 "-" "Wget" "-"
127.0.0.1 - - [19/Sep/2026:13:01:47 +0000] "GET / HTTP/1.1" 200 225 "-" "Wget" "-"
172.17.0.1 - - [19/Sep/2026:13:02:14 +0000] "GET / HTTP/1.1" 200 225 "-" "curl/8.5.0" "-"
127.0.0.1 - - [19/Sep/2026:13:02:17 +0000] "GET / HTTP/1.1" 200 225 "-" "Wget" "-"
127.0.0.1 - - [19/Sep/2026:13:02:47 +0000] "GET / HTTP/1.1" 200 225 "-" "Wget" "-"
etudiant@srv-vc01:~/inf6202/chap3/campus-web$
```

```
#7 DONE 1.0s
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker image ls uam-campus-web
                                                                                                    i Info →  U In Use
IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
uam-campus-web:1.0   29b334b43fd0       93.6MB         26.3MB
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker image inspect uam-campus-web:1.0 --format 'id={{.Id}} taille={{.Size}}'
id=sha256:29b334b43fd0d886e235443fb2dc118d165685237ce5b46bc8e21427f794f95c taille=93570358
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker rm -f campus-web 2>/dev/null || true
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker run -d --name campus-web -p 127.0.0.1:8080:80 uam-campus-web:1.0
473dd85e9d64b2c36f49f47c8ecd036dce29077a743adc8436843556ddc9ea2a
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker ps --filter name=campus-web
CONTAINER ID   IMAGE                COMMAND                  CREATED          STATUS                    PORTS                      NAMES
473dd85e9d64   uam-campus-web:1.0   "/docker-entrypoint.…"   48 seconds ago   Up 47 seconds (healthy)   127.0.0.1:8080->80/tcp   campus-web
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ curl fsS http://127.0.0
curl: (6) Could not resolve host: fsS
curl: (7) Failed to connect to 127.0.0.0 port 80 after 2 ms: Couldn't connect to server
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ curl fsS http://127.0.0.1:8080
curl: (6) Could not resolve host: fsS
<!doctype html>
<html lang="fr">
<meta charset="utf-8">
<title>UAM Campus Web</title>
<h1>UAM - INF6202</h1>
<p>Etudiant : Mamadou_Aliou_sow - 501866</p>
<p>Une image est un model ; un conteneur est une instance.</p>
</html>
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker logs --tail 20 campus-web
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/09/19 12:59:18 [notice] 1#1: using the "epoll" event method
2026/09/19 12:59:18 [notice] 1#1: nginx/1.31.6
2026/09/19 12:59:18 [notice] 1#1: built by gcc 15.2.0 (Alpine 15.2.0)
2026/09/19 12:59:18 [notice] 1#1: OS: Linux 6.8.0-139-generic
2026/09/19 12:59:18 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:524288
2026/09/19 12:59:18 [notice] 1#1: start worker processes
2026/09/19 12:59:18 [notice] 1#1: start worker process 30
2026/09/19 12:59:18 [notice] 1#1: start worker process 31
127.0.0.1 - - [19/Sep/2026:12:59:46 +0000] "GET / HTTP/1.1" 200 225 "-" "Wget" "-"
127.0.0.1 - - [19/Sep/2026:13:00:16 +0000] "GET / HTTP/1.1" 200 225 "-" "Wget" "-"
127.0.0.1 - - [19/Sep/2026:13:00:46 +0000] "GET / HTTP/1.1" 200 225 "-" "Wget" "-"
127.0.0.1 - - [19/Sep/2026:13:01:17 +0000] "GET / HTTP/1.1" 200 225 "-" "Wget" "-"
127.0.0.1 - - [19/Sep/2026:13:01:47 +0000] "GET / HTTP/1.1" 200 225 "-" "Wget" "-"
172.17.0.1 - - [19/Sep/2026:13:02:14 +0000] "GET / HTTP/1.1" 200 225 "-" "curl/8.5.0" "-"
127.0.0.1 - - [19/Sep/2026:13:02:17 +0000] "GET / HTTP/1.1" 200 225 "-" "Wget" "-"
127.0.0.1 - - [19/Sep/2026:13:02:47 +0000] "GET / HTTP/1.1" 200 225 "-" "Wget" "-"
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker inspect campus-web --format 'santé={{if .State.Health}}{{.State.Health.Status}}{{else}}absente{{end}}'
santé=unhealthy
etudiant@srv-vc01:~/inf6202/chap3/campus-web$
```

G-Provocation d'un défaut et mise à jour

- **Analyse de l'image dégradée (`:defaut`) :** Déplacer le fichier `index.html` n'a pas bloqué la compilation de l'image `:defaut`. Nginx s'appuie sur une configuration statique générique. Cela prouve qu'un build réussi (`FINISHED`) ne valide pas la conformité fonctionnelle du service rendu.

```
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ mv site/index.html site/index.txt
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker build -t uam-campus-web:defaut .
[+] Building 2.2s (7/7) FINISHED                                                
```

- **Immuabilité et isolation des couches (`:1.1`) :** La modification textuelle via la commande `sed` a modifié le contenu du répertoire `site/`. Lors du build de la version `1.1`, Docker a détecté ce changement, a invalidé le cache pour l'étape COPY et a généré une nouvelle couche. Les identifiants d'images (IDs) obtenus sont bien distincts, ce qui confirme l'immuabilité des images Docker : tout changement de code source produit une empreinte cryptographique unique.

```
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ mv site/index.txt site/index.html
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sed -i 's/UAM - INF6202 - version 1.1/' site/index.html
sed: -e expression #1, char 30: unterminated `s' command
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sed -i 's/UAM - INF6202 - version 1.1/' site/index.html
sed: -e expression #1, char 30: unterminated `s' command
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sed -i 's/UAM - INF6202/UAM - INF6202 - version 1.1/' site/index.html
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker build -t uam-campus-web:1.1 .
[+] Building 2.1s (7/7) FINISHED                                                
```

```
 => => unpacking to docker.io/library/uam-campus-web:1.1                                                    0.1s
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker image ls uam-campus-web

                                                                                          I Info →  U In Use
IMAGE                ID           DISK USAGE   CONTENT SIZE   EXTRA
uam-campus-web:1.0   29b334b43fd0       93.6MB         26.3MB   U
uam-campus-web:1.1   0803b6645b03       93.6MB         26.3MB
uam-campus-web:defaut   480010e3e870       93.6MB         26.3MB
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker image inspect uam-campus-web:1.0 uam-campus-web:1.1 --format '{{.Repotags}} {{.Id}}'

template parsing error: template: :1:2: executing "" at <.Repotags>: map has no entry for key "Repotags"
template parsing error: template: :1:2: executing "" at <.Repotags>: map has no entry for key "Repotags"
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker image inspect uam-campus-web:1.0 uam-campus-web:1.1 --format '{{.RepoTags}} {{.Id}}'
[uam-campus-web:1.0] sha256:29b334b43fd0d886e235443fb2dc118d165685237ce5b46bc8e21427f794f95c
[uam-campus-web:1.1] sha256:0803b6645b03b749b9ac5a5afc0b683887bf53c951759e9ace81cac6b9106a82
etudiant@srv-vc01:~/inf6202/chap3/campus-web$
```

H-Nettoyage et Archivage du projet:

- **Libération des ressources :** L'arrêt et la suppression du conteneur `campus-web` valident le cycle de vie du service. L'environnement local de la VM est laissé propre sans conteneurs orphelins actifs.
- **Intégrité de l'archive de livraison :** La commande `tar` a encapsulé l'ensemble du contexte (`Dockerfile`, `.dockerignore`, et le dossier `site/`) dans le fichier compressé `campus-web-501866.tgz` en excluant les fichiers de logs lourds. L'empreinte cryptographique SHA256 générée garantit l'intégrité de la livraison pour la soumission Moodle, prouvant qu'aucun fichier n'a été corrompu ou modifié après sa cré

```
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker stop campus-web
campus-web
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker rm campus-web
campus-web
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ tar --exclude='build-*.log' -czf ../campus-web-501866.tgz .
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sha256sum ../campus-web-501866.tgz
5458c3844c040fbfe8732b658eac9c396aa6dae37c1d698f616ebd079f1c04ae  ../campus-web-501866.tgz
etudiant@srv-vc01:~/inf6202/chap3/campus-web$
```

Conclusion Tp5:

Ce cinquième travail pratique a permis de valider la **maîtrise du processus de construction et de gestion d'images Docker personnalisées** sur notre serveur `srv-vc01`.

Les objectifs clés de ce TP ont tous été atteints :

- **Industrialisation et Sécurité :** L'écriture d'un `Dockerfile` documenté combiné à un fichier `.dockerignore` a permis de créer un contexte de build optimisé, léger et sécurisé, évitant l'inclusion de fichiers résiduels ou sensibles.
- **Gestion du cycle de vie et Immuabilité :** Les builds successifs (versions `1.0`, `1.1` et `defaut`) ont mis en évidence le mécanisme de mise en cache de Docker et l'immuabilité des couches. Toute modification du code source produit une empreinte cryptographique unique (ID d'image distinct).
- **Observabilité et Robustesse :** L'intégration de l'instruction `HEALTHCHECK` a permis de surveiller l'état de santé fonctionnel interne du service en temps réel, démontrant qu'un conteneur actif n'est pas systématiquement synonyme de conteneur sain.

  Le projet a été nettoyé et correctement archivé sous forme de livrable (`campus-web-501866.tgz`). Notre environnement est désormais prêt pour aborder le **TP 6**, qui portera sur la multi-contenerisation et l'orchestration de services.

3-Tp6:Réseau, volume et persistance d'un service Docker

A-Nettoyer les noms sans supprimer les données:

```
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker ps -a
[sudo] password for etudiant:
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker rm -f campus-check 2>/dev/null || true
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
00a8cdf6d10f   bridge    bridge    local
9ee8b8cb2bd0   host      host      local
1d2de9fffe36   none      null      local
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker volume ls
DRIVER    VOLUME NAME
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker image inspect uam-campus-web:1.0 --format 'image={{.Id}}'
image=sha256:29b334b43fd0d886e235443fb2dc118d165685237ce5b46bc8e21427f794f95c
etudiant@srv-vc01:~/inf6202/chap3/campus-web$
```

B-Créer le réseau utilisateur

Création de `campus-net` : Le réseau est initialisé avec le driver `bridge`. Il fournit un domaine de diffusion isolé et un résolveur DNS interne propre à Docker.

```
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker network create --driver bridge campus-net
03efb59bc5def3f0abea4b481f9c1ca088ffe80ceae7954f2ad06b4af207922a
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker network inspect campus-net --format 'nom={{.Name}} driver={{.Driver}} portée={{.Scope}}'
```

C-Créer le volume et lancer campus-web:

- **Montage de volume :** Le volume nommé `campus-content` est correctement attaché à la racine web (`/usr/share/nginx/html`). Docker y a recopié automatiquement la page `index.html` personnalisée issue de l'image lors du premier montage.
- **Disponibilité :** L'accès via `curl` sur le port `8080` de l'hôte confirme que le routage réseau et le stockage fonctionnent de concert.

```
nom=campus-net driver=bridge portée=local
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker volume create campus-content
campus-content
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker volume inspect campus-content
[
    {
        "CreatedAt": "2026-09-19T17:49:47Z",
        "Driver": "local",
        "Labels": null,
        "Mountpoint": "/var/lib/docker/volumes/campus-content/_data",
        "Name": "campus-content",
        "Options": null,
        "Scope": "local"
    }
]
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo_
```

```
        "CreatedAt": "2026-09-19T17:49:47Z",
        "Driver": "local",
        "Labels": null,
        "Mountpoint": "/var/lib/docker/volumes/campus-content/_data",
        "Name": "campus-content",
        "Options": null,
        "Scope": "local"
    }
]
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker run -d --name campus-web --network campus-net --mount type=volume,src=campus-content,dst=/usr/share/ng
inx/html -p 127.0.0.1:8080:80 uam-campus-web:1.0
fbefc67a3fde2e7043e181edabff1c385190346e086984c14f65422681c18be8
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker inspect campus-web --format 'réseaux={{json .NetworkSettings.Networks}} montages={{json .Mounts}}'
réseaux={"campus-net":{"IPAMConfig":null,"Links":null,"Aliases":null,"DriverOpts":null,"GwPriority":0,"NetworkID":"03efb59bc5def3f0abea4b481f9c1ca088ffe80ceae79
54f2ad06b4af207922a","EndpointID":"13ee8131c98f36ceb6f042ac7b2755b3fe6f8c130a7756c86342d984e6feec71","Gateway":"172.18.0.1","IPAddress":"172.18.0.2","MacAddress
":"ba:87:8d:40:08:dc","IPPrefixLen":16,"IPv6Gateway":"","GlobalIPv6Address":"","GlobalIPv6PrefixLen":0,"DNSNames":["campus-web","fbefc67a3fde"]}} montages=[{"Ty
pe":"volume","Name":"campus-content","Source":"/var/lib/docker/volumes/campus-content/_data","Destination":"/usr/share/nginx/html","Driver":"local","Mode":"z","
RW":true,"Propagation":""}]
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ curl -fsS http://127.0.0
curl: (7) Failed to connect to 127.0.0.0 port 80 after 1 ms: Couldn't connect to server
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ curl -fsS http://127.0.0.1:8080
<!doctype html>
<html lang="fr">
<meta charset="utf-8">
<title>UAM Campus Web</title>
<h1>UAM - INF6202</h1>
<p>Etudiant : Mamadou_Aliou_sow - 501866</p>
<p>Une image est un model ; un conteneur est une instance.</p>
</html>
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ _
```

D-Prouver la communication par nom:

- **Résultat du test positif :** Le conteneur éphémère `campus-check` parvient à joindre et afficher la page de `campus-web` en utilisant simplement son nom comme URL (http://campus-web/). Cela prouve que Docker intègre un **serveur DNS embarqué** qui résout automatiquement les noms des conteneurs connectés au même réseau utilisateur.
- **Résultat du test négatif :** La seconde commande échoue et affiche la chaîne `"échec attendu hors de campus-net"`. Cet échec est normal et attendu : le résolveur DNS interne de Docker et l'isolation réseau **interdisent la résolution par nom** pour tout conteneur externe rattaché au réseau `bridge` par défaut.

```
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker run --rm --name campus-check network campus-net alpine:3.20 wget -qO- http://campus-web/
Unable to find image 'network:latest' locally
docker: Error response from daemon: pull access denied for network, repository does not exist or may require 'docker login'

Run 'docker run --help' for more information
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker run --rm --name campus-check --network campus-net alpine:3.20 wget -qO- http://campus-web/
Unable to find image 'alpine:3.20' locally
3.20: Pulling from library/alpine
25f1d6b1951a: Pull complete
3a030ca3f633: Download complete
d4050d56ebf2: Download complete
Digest: sha256:d9e853e87e55526f6b2917df91a2115c36dd7c696a35be12163d44e6e2a4b6bc
Status: Downloaded newer image for alpine:3.20
<!doctype html>
<html lang="fr">
<meta charset="utf-8">
<title>UAM Campus Web</title>
<h1>UAM - INF6202</h1>
<p>Etudiant : Mamadou_Aliou_sow - 501866</p>
<p>Une image est un model ; un conteneur est une instance.</p>
</html>
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker run --rm alpine:3.20 wget -T 3 -qO- http://campus-web/ || echo 'échec attendu hors de campus-net'
wget: bad address 'campus-web'
échec attendu hors de campus-net
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ _
```

E-Écrire une preuve dans le volume:

- **Création de la preuve :** La commande `docker exec` a injecté un fichier `preuve.txt` contenant le matricule **501866** et l'horodatage système directement dans le stockage du serveur. Le fichier est parfaitement accessible depuis l'hôte via la commande `curl` ainsi que depuis le réseau interne `campus-net`.

```
echec attendu hors de campus-net
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker exec campus-web sh -c 'printf "matricule=501866\ndate=%s\n" "$(date -Iseconds)" > /usr/share/nginx/htm
l/preuve.txt'
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ curl -fsS http://127.0.0
curl: (7) Failed to connect to 127.0.0.0 port 80 after 2 ms: Couldn't connect to server
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ curl -fsS http://127.0.0.1:8080
<!doctype html>
<html lang="fr">
<meta charset="utf-8">
<title>UAM Campus Web</title>
<h1>UAM - INF6202</h1>
<p>Etudiant : Mamadou_Aliou_sow - 501866</p>
<p>Une image est un model ; un conteneur est une instance.</p>
</html>
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker run --rm --network campus-net alpine:3.20 wget -qO- http://campus-web/preuve.txt
matricule=501866
date=2026-09-19T18:22:35+00:00
etudiant@srv-vc01:~/inf6202/chap3/campus-web$
```

F-Validation de la persistance après suppression :

- **Survie des données :** Bien que le conteneur d'origine ait été brutalement supprimé (`docker rm -f`), le second `curl` exécuté sur le nouveau conteneur affiche toujours le matricule et la date initiale.
- **Conclusion pédagogique :** Le cycle de vie d'un conteneur est éphémère, mais les données persistent car elles résident dans le volume nommé autonome `campus-content`. La persistance est garantie par le remontage du volume au même chemin d'accès, indépendamment de l'identité du conteneur.

```
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker rm -f campus-web
campus-web
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker ps -a --filter name=campus-web
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker volume inspect campus-content
[
    {
        "CreatedAt": "2026-09-19T17:49:47Z",
        "Driver": "local",
        "Labels": null,
        "Mountpoint": "/var/lib/docker/volumes/campus-content/_data",
        "Name": "campus-content",
        "Options": null,
        "Scope": "local"
    }
]
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker run -d --name campus-web --network campus-net --mount type=volume,src=campus-content,dst=/usr/share/ng
inx/html -p 127.0.0.1:8080/80 uam-campus-web:1.0
docker: invalid proto: 80

Run 'docker run --help' for more information
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ sudo docker run -d --name campus-web --network campus-net --mount type=volume,src=campus-content,dst=/usr/share/ng
inx/html -p 127.0.0.1:8080:80 uam-campus-web:1.0
77b852333a7c680a5c68d8d53a9f5163f6d0003ac180341df1d608962373e4b7
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ curl -fsS http://127.0.0.1:8080
<!doctype html>
<html lang="fr">
<meta charset="utf-8">
<title>UAM Campus Web</title>
<h1>UAM - INF6202</h1>
<p>Etudiant : Mamadou_Aliou_sow - 501866</p>
<p>Une image est un model ; un conteneur est une instance.</p>
</html>
etudiant@srv-vc01:~/inf6202/chap3/campus-web$
```

G-Sauvgarde du contenue du volume:

- **Mécanisme utilisé :** La sauvegarde exploite un conteneur éphémère Alpine rattaché simultanément au volume nommé `campus-content` (en lecture seule pour la sécurité) et à un répertoire de l'hôte via un montage de type *bind*. L'outil `tar` encapsule les données directement vers l'hôte.
- **Résultat de l'archive :** Le listing de la commande `tar -tzf` prouve que l'archive contient bien la page d'accueil `index.html` et le fichier de traçabilité `preuve.txt`.
- **Sécurité et intégrité :** La commande `sha256sum` calcule l'empreinte unique de l'archive `campus-content.tgz`. Cela détache définitivement la sauvegarde du cycle de vie de Docker, assurant la résilience des données même en cas de suppression accidentelle du volume lui-même.

```
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ mkdir -p ~/inf6202/chap3/preuves-tp6
etudiant@srv-vc01:~/inf6202/chap3/campus-web$ cd ~/inf6202/chap3/preuves-tp6
etudiant@srv-vc01:~/inf6202/chap3/preuves-tp6$ sud[39549.423760] watchdog: BUG: soft lockup - CPU#1 stuck for 72s! [containerd-shim:11442]
o docker run --mpoio
unknown flag: --mpoio

Usage:  docker run [OPTIONS] IMAGE [COMMAND] [ARG...]

Run 'docker run --help' for more information
etudiant@srv-vc01:~/inf6202/chap3/preuves-tp6$ sudo docker run --mount type=volume,src=campus-content,dst=/data,readonly --mount type=bind,src="$PWD",dst=/backu
p alpine:3.20 tar -czf /backup/campus-content.tgz -C /data .
etudiant@srv-vc01:~/inf6202/chap3/preuves-tp6$ sudo chown "$USER:$USER" campus-content.tgz
etudiant@srv-vc01:~/inf6202/chap3/preuves-tp6$ tar -tzf campus-content.tgz
./
./index.html
./50x.html
./preuve.txt
etudiant@srv-vc01:~/inf6202/chap3/preuves-tp6$ sha256sum campus-content.tgz
eba15f24467ae1ab8c1d1369bf9bf9104ab3d5d3f494d1487c64334f83d4b138  campus-content.tgz
etudiant@srv-vc01:~/inf6202/chap3/preuves-tp6$ _
```

H-Inventaire final et remise.

- **P1 (Réseau `campus-net`) :** L'inspection du réseau utilisateur montre que Docker a automatiquement attribué l'adresse IP privée **`172.18.0.2/16`** au conteneur `campus-web`. La portée locale du bridge isole efficacement le trafic.
- **P2 (Montage du Volume) :** L'inspection montre le montage de type `volume` associant la source `campus-content` de l'hôte vers la destination cible `/usr/share/nginx/html/` du conteneur en mode `rw` (lecture-écriture).
- **P6 (Contenu de l'archive de sauvegarde) :** La commande `tar -tzf` exécutée à l'étape 7 a listé avec succès `./index.html`, `./50x.html` et `./preuve.txt`. L'empreinte SHA256 générée (`eba15f24467...`) valide la conformité de la sauvegarde hors du volume Docker.

```
etudiant@srv-vc01:~/inf6202/chap3/preuves-tp6$ sudo docker ps --filter name=campus-web
[sudo] password for etudiant:
CONTAINER ID   IMAGE                 COMMAND                  CREATED       STATUS                     PORTS                     NAMES
77b852333a7c   uam-campus-web:1.0    "/docker-entrypoint.…"   2 hours ago   Up 2 hours (unhealthy)     127.0.0.1:8080->80/tcp    campus-web
etudiant@srv-vc01:~/inf6202/chap3/preuves-tp6$
etudiant@srv-vc01:~/inf6202/chap3/preuves-tp6$ sudo docker network inspect campus-net --format '{{json .Containers}}'
{"77b852333a7c680a5c68d8d53a9f5163f6d0003ac180341df1d608962373e4b7":{"Name":"campus-web","EndpointID":"b9d6354e71b3564ed0329c36258983e5d4087c2dc949c97e834ee966f
3c2a88f","MacAddress":"56:17:c2:74:4f:ab","IPv4Address":"172.18.0.2/16","IPv6Address":""}}
etudiant@srv-vc01:~/inf6202/chap3/preuves-tp6$ sudo volume inspect campus content
sudo: volume: command not found
etudiant@srv-vc01:~/inf6202/chap3/preuves-tp6$ sudo docker volume inspect campus content
[]
Error response from daemon: get campus: no such volume
Error response from daemon: get content: no such volume
etudiant@srv-vc01:~/inf6202/chap3/preuves-tp6$ sudo docker inspect campus-web --format 'ports={{json .NetworkSettings.ports}} montages={{json .Mounts}}'

template parsing error: template: :1:29: executing "" at <.NetworkSettings.ports>: map has no entry for key "ports"
etudiant@srv-vc01:~/inf6202/chap3/preuves-tp6$ sudo docker inspect campus-web --format 'ports={{json .NetworkSettings.Ports}} montages={{json .Mounts}}'
ports={"80/tcp":[{"HostIp":"127.0.0.1","HostPort":"8080"}]} montages=[{"Type":"volume","Name":"campus-content","Source":"/var/lib/docker/volumes/campus-content/
_data","Destination":"/usr/share/nginx/html","Driver":"local","Mode":"z","RW":true,"Propagation":""}]
etudiant@srv-vc01:~/inf6202/chap3/preuves-tp6$ curl -fsS http://127.0.0.1:8080
<!doctype html>
<html lang="fr">
<meta charset="utf-8">
<title>UAM Campus Web</title>
<h1>UAM - INF6202</h1>
<p>Etudiant : Mamadou_Aliou_sow - 501866</p>
<p>Une image est un model ; un conteneur est une instance.</p>
</html>
etudiant@srv-vc01:~/inf6202/chap3/preuves-tp6$
</html>
etudiant@srv-vc01:~/inf6202/chap3/preuves-tp6$ [46722.767020] watchdog: BUG: soft lockup - CPU#1 stuck for 46s! [containerd-shim:11450]
sudo docker inspect campus-web --format 'ports={{json .Netwvolume inspect campus-content
[
    {
        "CreatedAt": "2026-09-19T17:49:47Z",
        "Driver": "local",
        "Labels": null,
        "Mountpoint": "/var/lib/docker/volumes/campus-content/_data",
        "Name": "campus-content",
        "Options": null,
        "Scope": "local"
    }
]
```

Conclusion tp6

Ce sixième travail pratique a permis de valider la gestion avancée des ressources Docker à travers l'isolation réseau et la persistance des données :

- **Isolation Réseau :** La création du bridge personnalisé `campus-net` a démontré l'utilité du serveur DNS interne de Docker pour la communication par nom entre conteneurs pairs, tout en bloquant l'accès aux conteneurs externes.
- **Persistance Découplée :** L'utilisation du volume nommé `campus-content` a prouvé que la durée de vie des données est totalement indépendante de celle des conteneurs. La suppression complète suivit de la recréation du conteneur n'a provoqué aucune perte de données.
- **Stratégie de Sauvegarde :** L'exportation réussie des données dans une archive compressée `.tgz` sur l'hôte concrétise la distinction cruciale entre la persistance locale d'un volume et une véritable sauvegarde externe exportable.

CONCLUSION GENERALE

Ce cycle de travaux pratiques dédiés à la **Virtualisation et au Cloud Computing** a permis de valider la mise en production, l'industrialisation et l'orchestration locale d'infrastructures conteneurisées sur notre serveur virtuel `srv-vc01`.

Trois compétences majeures ont été acquises et démontrées :

- **Conteneurisation et Cycle de vie (TP 4) :** L'installation de Docker Engine depuis son dépôt officiel stable a permis de maîtriser les commandes fondamentales de gestion des conteneurs éphémères (`hello-world`) et des services en arrière-plan (`nginx`), tout en garantissant l'isolation des processus et la légèreté des ressources (3.18 Mo de RAM consommés).
- **Industrialisation par le code (TP 5) :** L'écriture d'un `Dockerfile` documenté et l'usage d'un `.dockerignore` ont mis en évidence le principe d'immuabilité des images Docker. L'intégration d'un HEALTHCHECK a illustré l'importance de l'observabilité fonctionnelle en démontrant qu'un conteneur actif n'est pas obligatoirement un conteneur sain.
- **Persistance et Réseaux isolés (TP 6) :** Le déploiement d'un réseau de type *bridge* utilisateur (`campus-net`) et d'un volume nommé (`campus-content`) a prouvé le découplage total entre le cycle de vie des données et celui de l'application. Les données de traçabilité ont survécu à la destruction complète du conteneur et ont été sécurisées à travers une archive externe d'intégrité (`.tgz`).

  En somme, ce triptyque de TP pose les bases indispensables de l'architecture micro-services et du Cloud Computing. L'environnement de la machine `srv-vc01` est sain, documenté, validé par des preuves rigoureuses et prêt pour des scénarios d'orchestration à plus grande échelle.